

MOVE ZEROES

```
class Solution:
```

```
    def moveZeroes(self, nums):
```

```
        j = 0
```

```
        for i in range(len(nums)):
```

```
            if nums[i] != 0:
```

```
                nums[j], nums[i] = nums[i], nums[j]
```

```
                j += 1
```

NIM GAME

```
class Solution:
```

```
    def canWinNim(self, n):
```

```
        return n % 4 != 0
```

WORD PATTERN

class Solution:

def wordPattern(self, pattern, s):

words = s.split()

if len(pattern) != len(words):

return False

p_to_w = {}

w_to_p = {}

for p, w in zip(pattern, words):

if p in p_to_w and p_to_w[p] != w:

return False

if w in w_to_p and w_to_p[w] != p:

return False

p_to_w[p] = w

w_to_p[w] = p

return True

RANGE SUM QUERY

```
class NumArray:
```

```
    def __init__(self, nums):
```

```
        self.prefix = [0]
```

```
        for num in nums:
```

```
            self.prefix.append(self.prefix[-1] + num)
```

```
    def sumRange(self, left, right):
```

```
        return self.prefix[right + 1] - self.prefix[left]
```

POWER OF THREE

```
class Solution:
```

```
    def isPowerOfThree(self, n):
```

```
        if n <= 0:
```

```
            return False
```

```
        while n % 3 == 0:
```

```
            n //= 3
```

```
        return n == 1
```

COUNTING BITS

```
class Solution:
```

```
    def countBits(self, n):
```

```
        ans = [0] * (n + 1)
```

```
        for i in range(1, n + 1):
```

```
            ans[i] = ans[i // 2] + (i % 2)
```

```
        return ans
```

POWER OF FOUR

class Solution:

def isPowerOfFour(self, n):

if n <= 0:

return False

while n % 4 == 0:

n //= 4

return n == 1

REVERSE STRING

class Solution:

def reverseString(self, s):

left = 0

right = len(s) - 1

while left < right:

s[left], s[right] = s[right], s[left]

left += 1

right -= 1